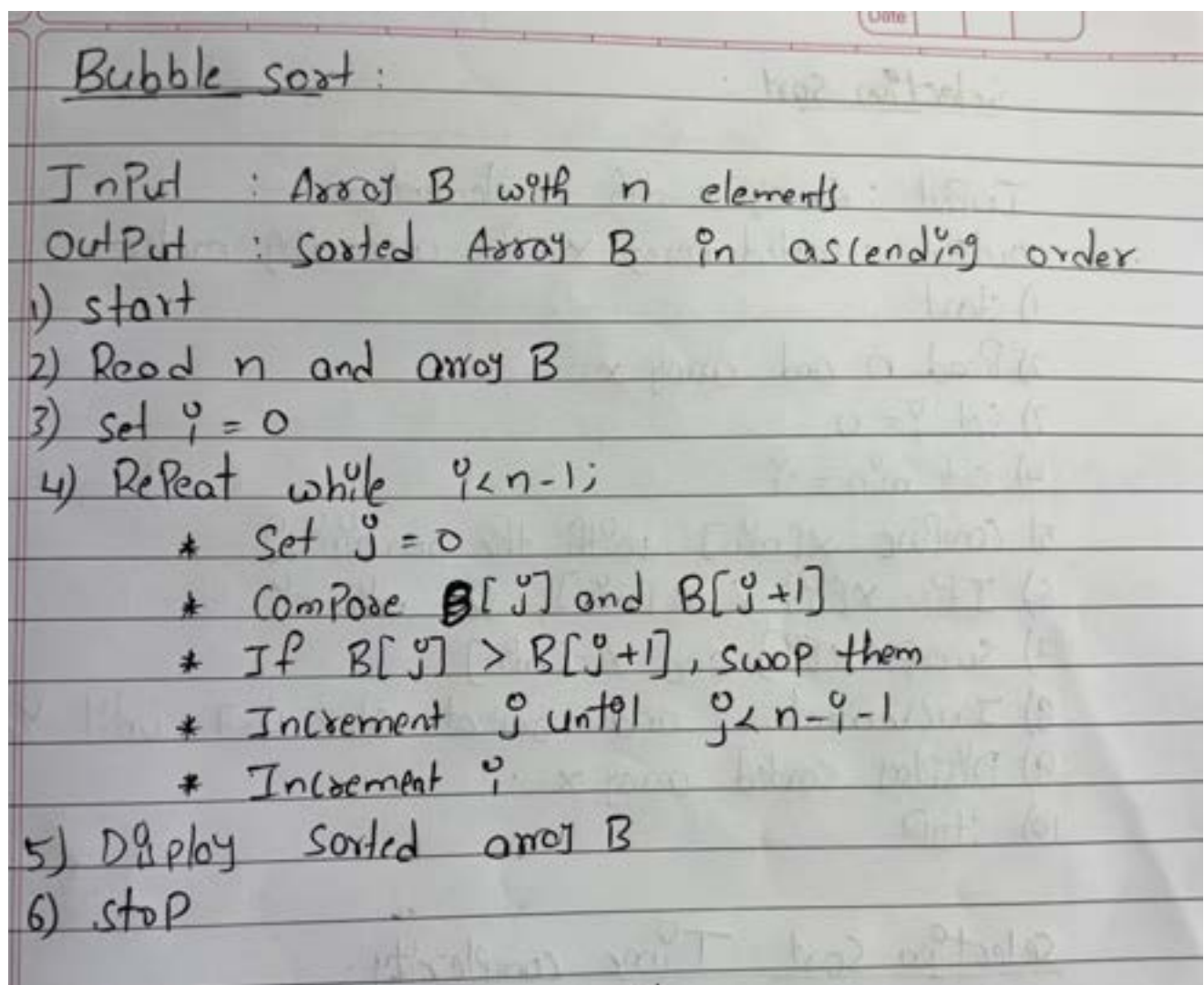


PRACTICAL: 01

Aim: To implementation and Time analysis of sorting algorithms.
Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

BUBBLE SORT :

Algorithm:



Program / Code:

```
#include<iostream>
using namespace std;
class bubblesort{
    public:
    int size,arr[100],i;
    void arrayInput(){
        cout<<"enter the size of array"<<endl;
        cin>>size;
        cout<<"enter array elements"<<endl;
        for(i=0;i<size;i++){
            cin>>arr[i];
        }
    }
    void PrintArray(){
        for(i=0;i<size;i++){
            cout<<arr[i];
        }
    }
    void BubbleLogic(){
        int j,temp;
        for(i=0;i<size-1;i++){
            for(j=0;j<size-1;j++){
                if(arr[j]>arr[j+1]){
                    temp=arr[j];
                    arr[j]=arr[j+1];
                    arr[j+1]=temp;
                }
            }
        }
    }
};

int main(){
    bubblesort bs;
    bs.arrayInput();
```

```
cout<<"\nbefore sorting"<<endl;
bs.PrintArray();
bs.BubbleLogic();
cout<<"\naftersorting"<<endl;
bs.PrintArray();
return 0;
}
```

Output:

```
C:\Users\NIKSHITH\DAA>bubble_sort.exe
enter the size of array
5
enter array elements
1
9
7
3
6

before sorting
19736
aftersorting
13679
C:\Users\NIKSHITH\DAA>
```

- **Best Case : $O(n)$**
- **Average Case : $O(n^2)$**
- **Worst Case : $O(n^2)$**

Time complexity :

Best Case:

Best case: (Bubble sort)

```
for (y=0; y<n-1; y++) {  
    swapped = false;  
    for (j=0; j<n-y-1; j++) {  
        if (arr[j] > arr[j+1]) {  
            swap (arr[j], arr[j+1]);  
            swapped = true;  
        }  
    }  
    if (swapped == false) {  
        break;  
    }  
}
```

$T(n) = 1 + 1 + (n-1) + (n-1) + 0 + 0 + 1 + 1 = 2n + 3$
 $\therefore$ We take highest order term
 $T(n) = O(n)$

Average and Worst Case :

Bubble-Sort Time Complexity

```
for (i=0; i<n-1; i++) {  
    for (j=0; j<n-1; j++) {  
        if (arr[j] > arr[j+1]) {  
            swap (arr[j] arr[j+1]);  
        }  
    }  
}
```

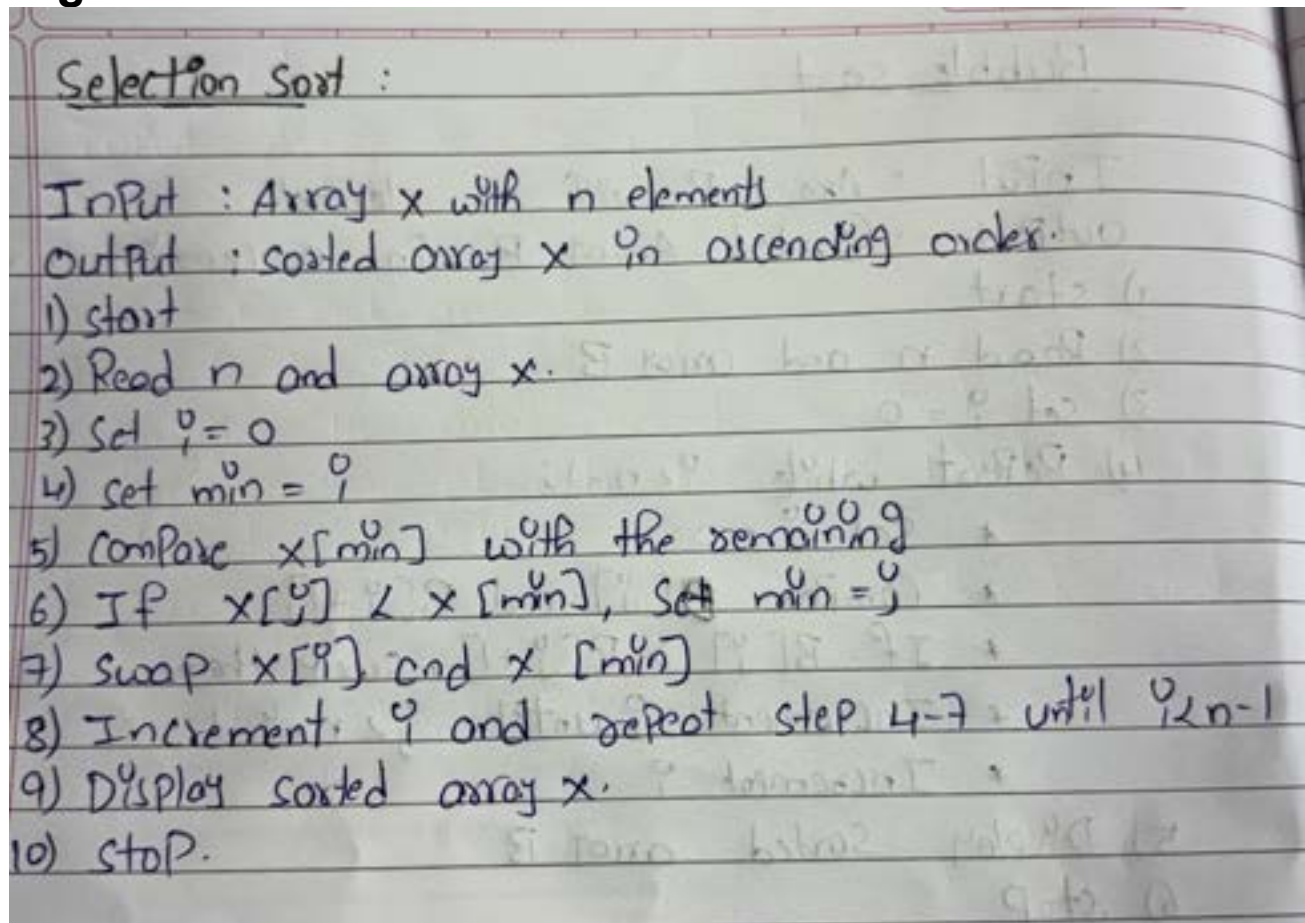
$T(n) = (n-1) + (n-2) + \dots + 1$
 $\Rightarrow (n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2} = \frac{n^2-n}{2}$

$\therefore$ We take highest

$T(n) = O(n^2)$

SELECTION SORT :

Algorithm:



Program / Code:

```
#include<iostream>
using namespace std;
class selectionsort{
public:
    int size,arr[100],i;
    void arrayInput(){
        cout<<"enter the size of array"<<endl;
        cin>>size;
```

```
cout<<"enter array elements"<<endl;
    for(i=0;i<size;i++){

cin>>arr[i];
    }
}

void PrintArray(){
    for(i=0;i<size;i++){
        cout<<arr[i];
    }
}

void selectionlogic(){
    int j,temp;
    for(i=0;i<size-1;i++){
        int min=i;
        for(j=i+1;j<size;j++){
            if(arr[j]<arr[min]){
                min=j;
            }
        }
        if(min!=i){
            temp=arr[i];
            arr[i]=arr[min];
            arr[min]=temp;
        }
    }
};

int main(){
    selectionsort ss;
    ss.arrayInput();
    cout<<"\nbefore sorting"<<endl;
    ss.PrintArray();
    ss.selectionlogic();
    cout<<"\naftersorting"<<endl;
```

```
ss.PrintArray();  
    return 0;  
}
```

Output:

```
C:\Users\NIKSHITH\DAA>g++ "selection sort.cpp" -o selection_sort.exe && selection_sort.exe  
enter the size of array  
6  
enter array elements  
9  
7  
3  
1  
5  
2  
  
before sorting  
973152  
aftersorting  
123579  
C:\Users\NIKSHITH\DAA>
```

Time Complexity:

- **Best Case : $O(n^2)$**
- **Average Case : $O(n^2)$**
- **Worst Case : $O(n^2)$**

Selection Sort Time Complexity:

```

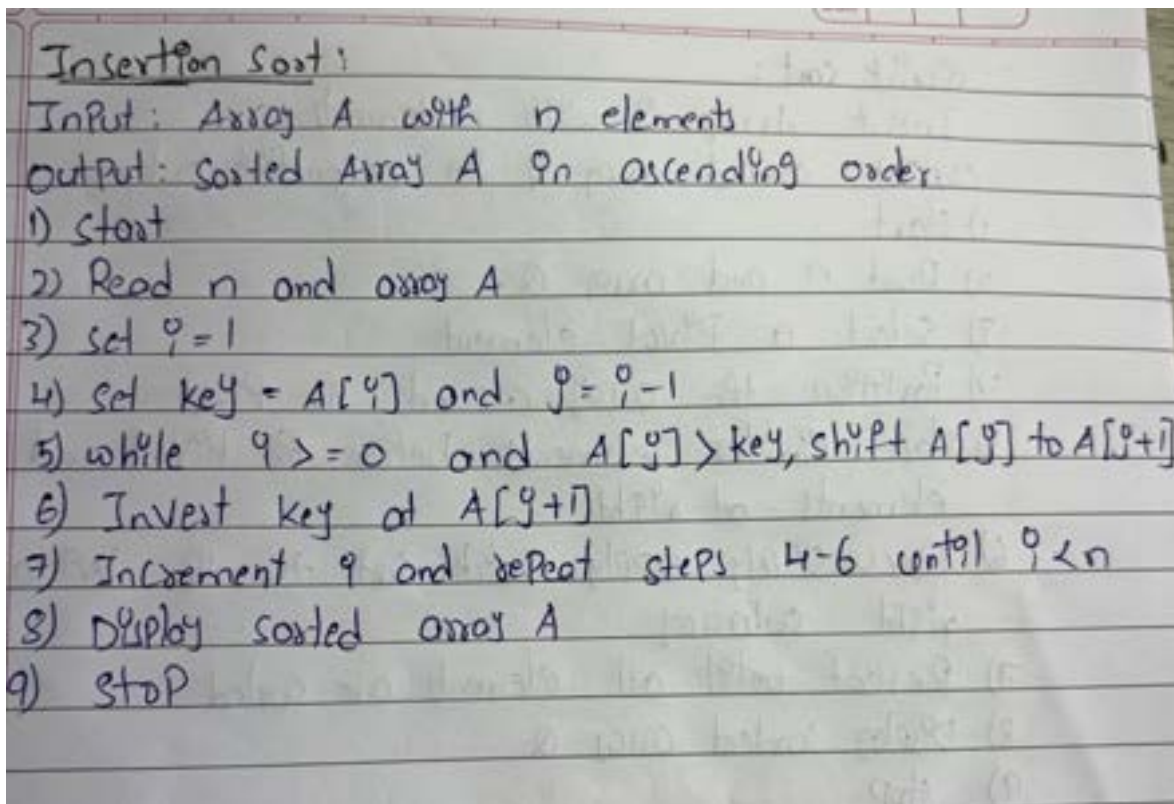
for (int i = 0; i < n - 1; i++) {
    int minIndex = i;
    for (int j = i + 1; j < n; j++) {
        if (arr[j] < arr[minIndex]) {
            minIndex = j;
        }
    }
    int temp = arr[i];
    arr[i] = arr[minIndex];
    arr[minIndex] = temp;
}

```

Total:
 $n + n - 1 + \dots + 1 = \frac{n(n-1)}{2} = O(n^2)$
 $T(n) = O(n^2)$

3) INSERTION SORT

Algorithm:



Program / Code:

```
#include<iostream>
using namespace std;
class insertionsort{
public:
    int size,arr[100],i;
    void arrayInput(){
        cout<<"enter the size of array"<<endl;
        cin>>size;
        cout<<"enter array elements"<<endl;
        for(i=0;i<size;i++){
            cin>>arr[i];
        }
    }
}
```

```
void PrintArray(){
    for(i=0;i<size;i++){
        cout<<arr[i];
    }
}

void insertionlogic(){
    int temp,j;
    for(i=1;i<size;i++){
        temp=arr[i];
        j=i-1;
        while(j>=0&&arr[j]>temp){
            arr[j+1]=arr[j];
            j--;
        }
        arr[j+1]=temp;
    }
}

};

int main(){
    insertionsort ls;
    ls.arrayInput();
    cout<<"\nbefore sorting"<<endl;
    ls.PrintArray();
    ls.insertionlogic();
    cout<<"\naftersorting"<<endl;
    ls.PrintArray();
    return 0;
}
```

Output :

```
C:\Users\NIKSHITH\DAA>g++ "Insertion sort.cpp" -o insertion_sort.exe && insertion_sort.exe
enter the size of array
7
enter array elements
9
2
5
1
7
6
4

before sorting
9251764
aftersorting
1245679
C:\Users\NIKSHITH\DAA>
```

Time Complexity:

- **Best Case** : $O(n)$
- **Average Case** : $O(n^2)$
- **Worst Case** : $O(n^2)$

Best Case :

Insertion sort + Best Case

```
for (int i = 1; i < n; i++) {           → n-1
    int key = arr[i];                  → n-1
    while (j >= 0 & arr[j] > key) {      → 1
        arr[j+1] = arr[j];             → 0
        j--;
    }
    arr[j+1] = key;                     → n-1
}
```

$T(n) = 1 + 1 + 1 + \dots + 1 \text{ (n-1 times)} \Rightarrow n-1$

$$\boxed{T(n) = O(n)}$$

Average and Worst Case :

Insertion Sort Time Complexity

```
for (int i=1; i<n; i++) {  
    int key = arr[i];  
    while (j>=0 && arr[j] > key) {  
        arr[j+1] = arr[j];  
        j--;  
    }  
    arr[j+1] = key;  
}
```

Total : $1 + 2 + 3 + \dots + (n-1) \Rightarrow \frac{n(n-1)}{2}$

$T(n) = O(n^2)$

4) Merge Sort Algorithm :

Merge Sort Algorithm
Input: Array A with n elements
Output: Sorted array A in ascending order.

Step 1: Calculate sizes of temporary subarrays:
 $n_1 = \text{mid} - \text{left} + 1$
 $n_2 = \text{right} - \text{mid}$

Step 2: Declare temporary arrays L[n₁] and R[n₂]

Step 3:
For $i = 0$ to $n_1 - 1$: $L[i] = \text{arr}[\text{left} + i]$
For $j = 0$ to $n_2 - 1$: $R[j] = \text{arr}[\text{mid} + 1 + j]$

Step 4: Set index variables $i = 0$, $j = 0$ and $k = \text{left}$

Step 5: While $i < n_1$ and $j < n_2$:

- If $L[i] \leq R[j]$ then:
 $\text{arr}[k] = L[i]$
 Increment i
- Else
 $\text{arr}[k] = R[j]$
 Increment j
- Increment k

Step 6:
While $i < n_1$: copy remaining elements of L[i] to arr[k],
 Increment i and k .
While $j < n_2$: copy remaining elements of R[j] to arr[k],
 Increment j and k .

Step 7: Stop.

Program / Code :

```
#include <iostream>
#include <vector>

void merge(std::vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    std::vector<int> L(n1);
    std::vector<int> R(n2);
    for (int i = 0; i < n1; ++i) {
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; ++j) {
        R[j] = arr[mid + 1 + j];
    }

    int i = 0;
    int j = 0;
    int k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            ++i;
        } else {
            arr[k] = R[j];
            ++j;
        }
        ++k;
    }

    while (i < n1) {
        arr[k] = L[i];
        ++i;
        ++k;
    }
```

```
}  
    while (j < n2) {  
        arr[k] = R[j];  
        ++j;  
        ++k;  
    }  
}  
  
void mergeSort(std::vector<int>& arr, int left, int right) {  
    if (left < right) {  
        int mid = left + (right - left) / 2;  
  
        mergeSort(arr, left, mid);  
        mergeSort(arr, mid + 1, right);  
  
        merge(arr, left, mid, right);  
    }  
}  
  
int main() {  
    int n;  
    std::cout << "Enter the number of elements: ";  
    std::cin >> n;  
  
    if (n <= 0) {  
        return 0;  
    }  
  
    std::vector<int> arr(n);  
    std::cout << "Enter " << n << " integers: ";  
    for (int i = 0; i < n; ++i) {  
        std::cin >> arr[i];  
    }  
  
    mergeSort(arr, 0, n - 1);  
}
```

```
std::cout << "Sorted array: ";  
for (int i = 0; i < n; ++i) {  
    std::cout << arr[i] << " ";  
}  
std::cout << std::endl;  
  
return 0;  
}
```

Output :

```
C:\Users\NIKSHITH\DAAG>g++ "merge sort.cpp" -o merge_sort.exe && merge_sort.exe  
Enter the number of elements: 6  
Enter 6 integers: 9  
7  
2  
5  
1  
6  
Sorted array: 1 2 5 6 7 9
```

Time Complexity :

- **Best Case** : $O(n \log n)$
- **Average Case** : $O(n \log n)$
- **Worst Case** : $O(n \log n)$

Time Complexity Merge Sort:

```

void mergesort (int a[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mergesort (a, l, m); mergesort (a, m + 1, r);
        merge (a, l, m, r);
    }
}
  
```

merge sort recurrence:

$$T(n) = 2T(n/2) + O(n)$$

Compare with

$$T(n) = aT(n/b) + f(n)$$

$$a = 2, k = 1, b = 2$$

$$f(n) = n$$

$$p = 0$$

$$\log_2 2 = 1$$

$$k = \log_2 2$$

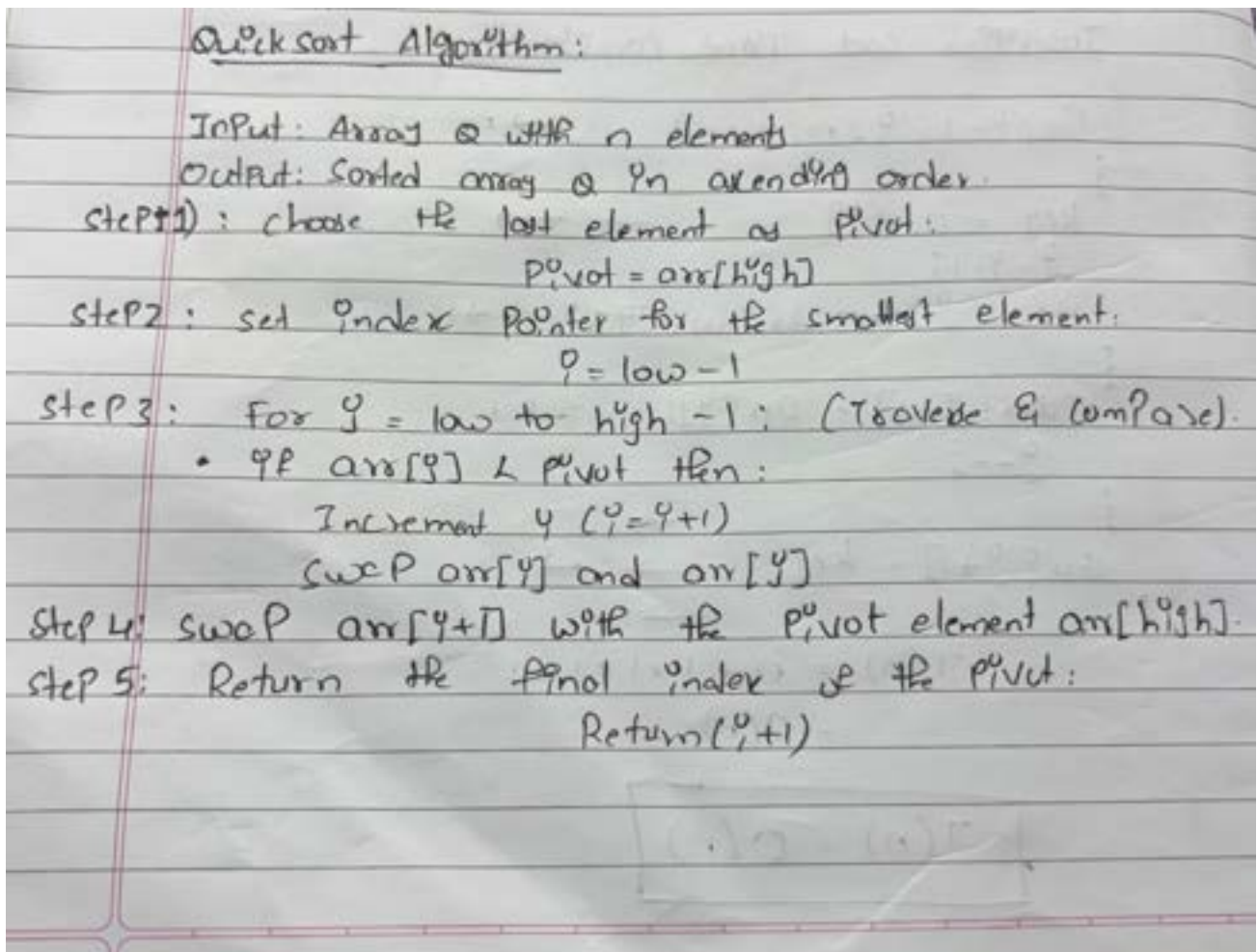
$$\text{so, here } f(n) = \log_b a$$

$$\text{so, } T(n) = n \log n$$

$$T(n) = O(n \log n)$$

5) Quick Sort :

Algorithm :



Program / Code :

```
#include<iostream>
using namespace std;
class Quicksort{
public:
    int size, arr[100];
    void arrayInput(){
        cout<<"enter the size of array"<<endl;
        cin>>size;
```



```
cout<<"enter array elements"<<endl;
    for(int i=0; i<size; i++){
        cin>>arr[i];
    }
}
void PrintArray(){
    for(int i=0; i<size; i++){
        cout<<arr[i]<<" ";
    }
    cout<<endl;
}
void QuickSortlogic(int lb, int ub){
    if(lb < ub){
        int pivot = arr[lb];
        int start = lb;
        int end = ub;
        while(start < end){

            while(start < ub && arr[start] <= pivot){
                start++;
            }
            while(arr[end] > pivot){
                end--;
            }
            if(start < end){
                swap(arr[start],arr[end]);
            }
        }
        swap(arr[lb],arr[end]);
        int loc = end;
        QuickSortlogic(lb, loc - 1);
        QuickSortlogic(loc + 1, ub);
    }
}
};
int main(){
```

```
Quicksort q;  
    q.arrayInput();  
    cout<<"\nBefore Sorting"<<endl;  
    q.PrintArray();  
    q.QuickSortlogic(0, q.size - 1);  
    cout<<"\nsorted array:"<<endl;  
    q.PrintArray();  
    return 0;  
}
```

Output :

```
C:\Users\NIKSHITH\DAAG>g++ "Quick sort.cpp" -o quick_sort.exe && quick_sort.exe  
enter the size of array  
5  
enter array elements  
9  
1  
6  
3  
7  
  
Before Sorting  
9 1 6 3 7  
  
sorted array:  
1 3 6 7 9
```

Time Complexity :

- **Best Case** : $O(n \log n)$
- **Average Case** : $O(n \log n)$
- **Worst Case** : $O(n^2)$

Best and Average Case :

Time Complexity of Quick sort:

```

void QuickSort (int a[], int low, int high) {
    if (low < high) {
        int p = Partition (a, low, high);
        QuickSort (a, low, p-1);
        QuickSort (a, p+1, high);
    }
}
  
```

Partition $O(n)$

Quick sort (left) + Quick sort (right)

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

By using master method.

$a = 2, b = 2, f(n) = n, k = 1, p = 0$

$$\log_2 2 = 1$$

By comparing k & $\log_b^a$ value $k = \log_b^a$

So,

$$T(n) = n \log n$$

$T(n) = n \log n \rightarrow$ Best / Average case.

$T(n) = O(n^2) \rightarrow$ Worst Case.

Worst Case :

Quick sort (Worst case):

```

void quicksort (int a[], int low, int high) {
    if (low < high) {
        int p = Partition (a, low, high);
        quicksort (a, low, p-1);
        quicksort (a, p+1, high);
    }
}
    
```

Partition = $O(n)$
 Quicksort (left) + Quicksort (right)

$$T(n) = T(0) + T(n-1) + O(n)$$

$$T(n) = T(n-1) + n$$

By substitution method:

$$T(n) = T(n-1) + n$$

$$T(n) = [T(n-2) + (n-1)] + n$$

$$T(n) = T(n-3) + (n-2) + (n-1) + n$$

$$\vdots$$

$$T(n) = 1 + 2 + 3 + \dots + n$$

$$T(n) = n(n+1) / 2 = (n^2 + n) / 2$$

$T(n) = O(n^2)$